

Arrays, Multidimensional Matrices and Memory Layout in C

[DEF] Definition to Remember

An array is a collection of elements of identical data types stored in contiguous memory locations, offering O(1) random access via index arithmetic.

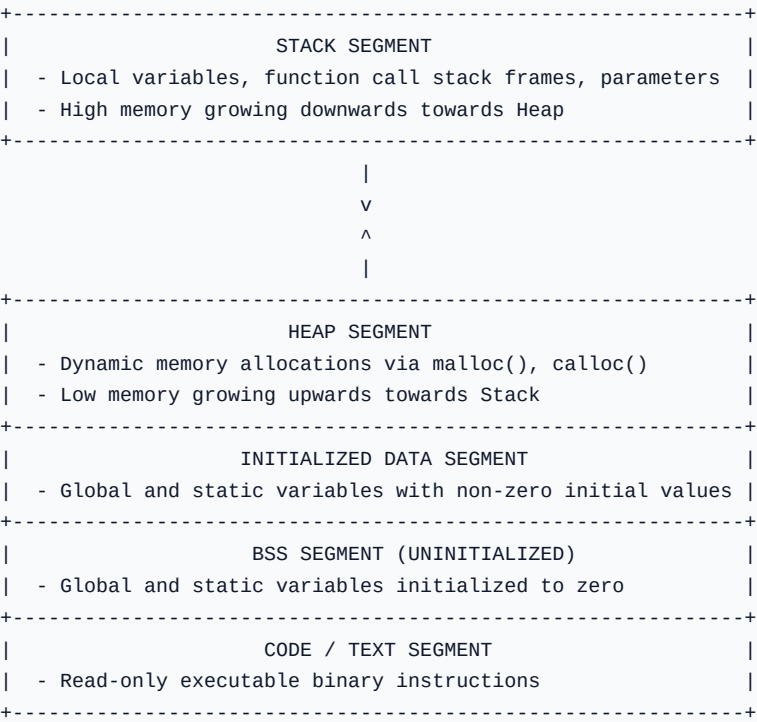
1. Concept Overview & Fundamentals

Arrays, Multidimensional Matrices and Memory Layout in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

2. Technical Architecture & Memory Mechanics

Memory Layout & Storage Organization



Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers	Stack: O(1) push/pop; Heap: O(1)
Type Checking	Static Typing	Compiler verifies types during compila	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memor	Direct memory dereference (O(1))
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops,	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: Arrays\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- * Operating System Kernels: Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.

* Em

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: array max min

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Create
elemen
*/

#include

int ma
/*
*
*
*
*
*
*
*
in
in
pr
fo
```

```
}  
  
ma  
mi  
  
fo  
  
  
  
}  
  
pr  
pr  
  
re  
  
}
```

Problem 2: array occurrences

Problem Description & Objective:

C Code Implementation:

Implem

```
/*  
Create  
of an  
*/  
  
#inclu  
  
int ma  
/*  
*  
*  
*  
*  
*  
*  
*  
  
in  
  
pr  
  
fo  
  
}  
  
pr  
sc  
  
fo  
  
}
```

```
pr
re
}
```

Problem 3: array palindrome

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Create
*/

#include

int ma
/*
*
*
*
*
*
*
*

in
in
pr

fo

}

fo

}

if

}

}

re
}
```

Problem 4: array sorted check

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Create
```

```
*/  
  
#inclu  
  
int ma  
/*  
*  
*  
*  
*  
*  
*  
  
in  
in  
  
pr  
  
fo  
  
}  
  
fo  
  
  
}  
  
if  
  
}  
  
}  
  
re  
}
```

Problem 5: array sum average

Problem Description & Objective:

C Code Implementation:

Implem

```
/*  
Create  
of all  
*/  
  
#inclu  
  
int ma  
/*  
*  
*  
*  
*  
*  
*  
  
in
```

```
in
fl

pr

fo

}

av

pr
pr

re
}
```

Problem 6: copy char array pointer

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Write
to cop
*/

#include

// Fun
void c
/*
*
*
*
*
*
*
*
wh

}

*c
}

int ma
ch

pr
sc

co

pr

re
}
```

Problem 7: delete array element

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Create
a spec
*/

#include

int ma
/*
*
*
*
*/
writes
*
*
*
*
*
in
in

pr
if

}

if

}

pr
fo

}

pr
if

}

fo

}

pr
```

```
fo
}
re
}
```

Problem 8: diagonal sum

Problem Description & Objective:

C Code Implementation:

Implem

```
/*
Create
*/
#include

int ma
/*
*
*
*
*
*
*
*

in
in
in
in

pr

fo

}

//
fo

}

//
fo

}

pr
pr

re
}
```

Problem 9: merge sorted arrays

Problem Description & Objective:

Implem

C Code Implementation:

```
/*
Create
*/

#include

int ma
/*
*
*
*
*
*
*
*/

in
in

pr

fo

}

pr

fo

}

i
j

wh

}

wh

}

wh

}

pr

fo
```

```
}  
re  
}
```

Problem 10: reverse array

Problem Description & Objective:

C Code Implementation:

Implem

```
/*  
Create  
*/  
  
#inclu  
  
int ma  
/*  
*  
*  
*  
*  
*  
*  
*  
  
in  
  
pr  
  
fo  
  
}  
  
pr  
  
fo  
  
}  
  
re  
}
```

Problem 11: search 2d array

Problem Description & Objective:

C Code Implementation:

Implem

```
/*  
Create  
*/  
  
#inclu  
  
int ma  
/*  
*  
*  
*  
*  
*  
*  
*  
  
in  
  
pr  
  
fo  
  
}  
  
pr  
  
fo  
  
}  
  
re  
}
```

```

*
*
*
in
in
pr
fo

}

pr
sc
fo

}

if
}

re
}

```

Problem 12: sum average 2d array

Problem Description & Objective:

C Code Implementation:

Implem

```

/*
Create
of all
*/

#include

int ma
/*
*
*
*
*
*
*
in
in
fl

```

6. Exam Scoring Points & Memory Revision

[MUST-WRITE] Must-Write Points

1. Contiguous memory calculation: Address of $\text{arr}[i] = \text{Base Address} + i * \text{sizeof}(\text{type})$.
2. Array indexing starts at 0 and ends at N-1. Accessing $\text{arr}[N]$ causes undefined behavior / memory corruption.
3. 2D Arrays: Stored in Row-Major order in memory (Row 0 elements followed by Row 1).
4. Passing arrays to functions decays them into pointers to the first element ($\text{int}^* \text{arr}$).
5. Standard operations: Traversal $O(N)$, Search $O(N)$, Reversal $O(N)$, Merge $O(N+M)$.

[RECALL] Quick Recall

Base Address + $(i * \text{size}) \rightarrow$ Contiguous Stack $\rightarrow$ Row-Major 2D $\rightarrow$ Pointer Decay $\rightarrow$ Bounds Check